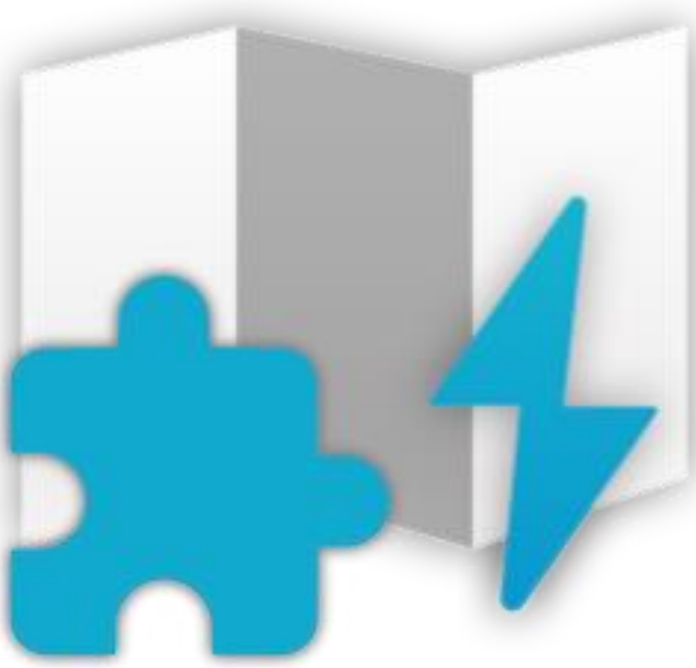

Input System Extension

Documentation

_____

Creator: Lucas Gomes Cecchini

Pseudonym: AGAMENOM

Overview

This document provides guidance on using the **Input System Extension** for **Unity**, a comprehensive framework designed to simplify input configuration, rebinding, and runtime input handling across multiple device types including keyboard, mouse, gamepad, and touch.

With this extension, you can:

• Easily customize and remap input controls directly from in-game menus, UI panels, or the editor interface.

• Use a centralized configuration asset to manage input settings, icon mappings, and persistence behavior.

• Perform runtime rebinding with full lifecycle management, including start, completion, cancellation, duplicate prevention, and real-time UI feedback.

• Automatically save and load input binding overrides using PlayerPrefs, ensuring settings persist between sessions.

• Display input bindings using either text or device-specific icons, with automatic switching based on the active controller (e.g., keyboard, Xbox, PlayStation).

• Reset individual bindings or all bindings at once, allowing users to quickly revert to default configurations.

• Utilize a flexible input event system that converts raw input into structured events (pressed, hold, released), enabling multiple systems to safely share the same input actions.

• Detect input activity across multiple device types to dynamically adapt UI and gameplay behavior.

• Quickly set up input UI elements using built-in editor tools and prefab creation utilities.

The Input System Extension provides a scalable and modular solution for input management, combining runtime rebinding, visualization, persistence, and event-driven input handling into a unified workflow.

Instructions

You can get more information from the Playlist on YouTube:

https://www.youtube.com/playlist?list=PL5hnfx09yM4Kqkhx0KHyUW0kWviPMTPCs

Script Explanations

Get Action Attribute

This script is essentially an **editor tool** that improves how you pick input actions inside the Unity Inspector. Instead of manually dragging references, it gives you a **dropdown list of actions** coming from a predefined input asset.
Let's break it down clearly.

🧠 Core Idea

The script introduces a tag ([GetAction]) that you can place on a field.
When Unity sees this tag, it replaces the normal field with a **dropdown menu listing all available input actions** from a central configuration.

📦 Main Components

1. Attribute: GetActionAttribute
This is the simplest part.
What it is:
A **marker**.
What it does:
It tells Unity:
"Hey, when drawing this field in the inspector, use a custom interface instead of the default one."
Important detail:

- It doesn't contain logic.
- It only exists so Unity can detect it.

2. Drawer: GetActionDrawer
This is where all the real work happens.
It is responsible for:

- Drawing the dropdown.

- Validating the data.
- Handling selection changes.

📊 **Variables Explained**

extensionData

- A cached reference to a configuration object.
- This object holds the **default input action asset**.
- It is loaded only once and reused (for performance).

⚙️ **Main Method: OnGUI**

This is the **heart of the system**.
It runs every time the inspector needs to draw the field.

Step-by-step behavior

1. Type validation

- Checks if the field is a reference type.
- If not → shows an error message.

👉 Why:
The system only works with input action references.

2. Load configuration

- If not already loaded, it fetches the configuration object.

👉 Why:
This object contains the input asset used as source.

3. Validate configuration

- If the config or asset is missing → shows error.

👉 Prevents:
Broken inspector UI.

4. Collect all actions

- Goes through all action maps.
- Extracts all valid actions.
- Builds a list.

👉 Result:
A flat list of selectable actions.

5. Handle empty case

- If no actions exist → warning is shown.

6. Read current value

- Gets the currently assigned reference.

7. Validate current selection

Checks:

- Does the selected action still exist in the asset?

If not:

- Marks it as **invalid**
- Builds a warning label like:
- [⚠ INVALID] MapName/ActionName

👉 This is very important:
It prevents silent errors when assets change.

8. Build dropdown list

Creates display names like:
Player/Jump
Player/Move
UI/Click

9. Determine selected index

- If invalid → put invalid item at the top.
- Otherwise → find correct index.

10. Draw UI

- Draws label.
- Draws dropdown (popup).

11. Show warning (if needed)

If invalid selection:

- Displays a warning box below the dropdown.

12. Handle user selection

When user changes selection:

Steps:

1. Adjust index if invalid item exists.
2. Get selected action.
3. Search for an existing reference in the asset.
4. Assign it to the field.
5. Mark object as modified.

👉 Important design choice:

It **does not create new references**, it reuses existing ones.

📏 **Second Method: GetPropertyHeight**

This controls how much vertical space the field uses.

Behavior:

Default:

- One line height.

If errors exist:

- Adds extra space for help boxes.

If invalid selection:

- Adds space for warning message.

🎯 **Usage**

How to use it

You simply mark a field like this:

[GetAction]

<your input action reference field>

What happens in the Inspector

Instead of:

- Dragging references manually

You get:

- A clean dropdown with all available actions

Benefits

✓ **Faster workflow**

No need to search assets manually.

✓ **Safer**

- Detects invalid references.
- Prevents broken links.

✓ **Centralized**

Everything comes from a single input asset.

✓ **Cleaner inspector**

User-friendly UI instead of raw references.

⚠ **Limitations / Requirements**

- Only works inside the editor (not in builds).
- Requires a configured input asset.
- Only works with compatible field types.

🧩 **Big Picture**

This script acts like a **bridge between your input system and your inspector UI**.
Without it:

- You manually manage references.

With it:

- You select actions like choosing options from a menu.

Binding Id Attribute

This script is another **editor enhancement tool**, but instead of selecting an *action*, it lets you select a **specific binding inside an action**.
In simple terms:

👉 You pick an input action (like *Jump*),

👉 then this script lets you pick *which key/button* (binding) of that action you want.

🧠 **Core Idea**

The script links **two fields inside the same object**:

- One field = the **input action**
- Another field = the **binding ID (a specific control like keyboard key, gamepad button, etc.)**

The attribute [BindingId("actionFieldName")] creates that connection.

📦 **Main Components**

1. Attribute: BindingIdAttribute
What it is:
A **link descriptor**.
Variable:

- **actionFieldName**
  - Stores the name of another field in the same object.
  - That field must contain the input action reference.

Constructor (how it's created)

When you write:
[BindingId("myAction")]
It stores:
actionFieldName = "myAction"

👉 This tells the system:
“Go look at that other field to know where to get bindings from.”

🧩 **How the Link Works**

This is the key concept:
The script does not directly store the action.
Instead, it:

1. Reads the current field’s location.
2. Replaces its name with the target field name.
3. Finds the corresponding action field dynamically.

👉 This allows it to work even inside nested objects and arrays.

🎨 **Drawer: BindingIdDrawer**

This is the part that builds the **dropdown of bindings**.

⚙ **Main Method: OnGUI**

This method runs every time the inspector draws the field.

Step-by-step behavior

1. Validate attribute
Checks if the attribute exists and is correct.
If not:
- Falls back to normal field drawing.

2. Find the linked action field
- Gets the current field path.
- Replaces its name with the target action field name.
- Searches for that field in the serialized object.

Possible problem:
If not found → shows error:
Action field 'X' not found.

3. Get the action
From the linked field:
- Extracts the action reference.
- Then gets the actual action.

Possible problem:
If no action is assigned → warning:
No InputAction selected.

4. Get bindings
- Retrieves all bindings from the action.
- Counts how many exist.

👉 Each binding = a specific control
Examples:
- Space key
- Gamepad A button
- Mouse click

5. Prepare dropdown data
Creates:
- **options** → what the user sees
- **optionValues** → actual binding IDs (internal values)

Also:
- Reads current selected binding ID
- Tracks which index is currently selected

6. Build display text for each binding
This is one of the most important parts.
For each binding:

a) Get binding ID
- Unique identifier (stored as text)

b) Determine display style
Uses options to:
- Show full readable names
- Ignore overrides
- Include device names if needed

c) Generate readable text
Examples:
Keyboard Space

Gamepad Button South
Mouse Left Button

d) Handle composite bindings
If the binding is part of something like movement:
Example:
WASD input:
  Up: W
  Down: S
It becomes:
Up: W
Down: S

e) Prevent UI issues
Replaces / with \
👉 Why:
Unity uses / to create submenus, which would break the dropdown.

f) Add control scheme info
If binding belongs to schemes (like Keyboard/Gamepad):
Example:
Space (Keyboard & Mouse)
A Button (Gamepad)

g) Store results
- Save display text in options
- Save binding ID in optionValues

h) Detect current selection
Compares stored ID with each binding ID
→ sets the selected index

7. Draw UI
- Starts property drawing
- Draws label (with tooltip)
- Draws dropdown with all binding options

8. Handle selection change
If user picks a different binding:
1. Update stored value with selected binding ID
2. Apply changes to serialized data
3. Mark object as modified (so Unity saves it)

🎯 **Usage**
How to use it
You define two fields:
Action reference field
Binding ID field (string)
Then link them:
[BindingId("actionFieldName")]

Example concept (not code-specific)
- Field A → contains "Jump action"
- Field B → contains "which key triggers Jump"

The dropdown in Field B will now show:
- Space
- Gamepad A
- etc.

✅ **What You Gain**

✓ **Precise control**

You don't just pick an action — you pick *how it is triggered*.

✓ **Dynamic UI**

Changes automatically based on the selected action.

✓ **Safe linking**

If the action changes, the binding list updates.

✓ **Human-readable**

Bindings are shown in a clean, understandable format.

⚠ **Limitations**

- Only works inside the editor.
- Requires a valid action reference.
- Stores binding IDs as text (not direct references).

🧩 **Big Picture**

This script complements the previous system:

- First script → choose **which action**
- This script → choose **which binding of that action**

Together, they give you a **fully controlled input selection workflow directly in the inspector**.

Input System Extension Prefab Creator

This script is another **editor-only utility**, but this time focused on **creating ready-to-use UI objects in your scene** with almost zero manual setup.

Think of it as:

👉 *A "spawn tool" for your input system UI prefabs.*

🧠 **What this script does (big picture)**

It adds options to Unity's top menu:

👉 **GameObject → Tools → Input System Extension → UI**

When you click one of those options, it will:

1. Find the correct prefab in your project
2. Instantiate it into the scene
3. Automatically place it correctly (under selected object or Canvas)
4. Create a Canvas + EventSystem if needed
5. Make the object editable
6. Select it and allow instant renaming

🧩 **Main Class**

InputSystemExtensionPrefabCreator

A static utility that:

- Adds menu options
- Handles prefab creation
- Manages UI setup automatically

⚙ **Core Concepts in the Script**

There are **4 main internal operations**:

1. **Create UI environment (Canvas + EventSystem)**
2. **Find prefab in project safely**
3. **Instantiate and parent it correctly**
4. **Finalize setup (undo, unpack, rename)**

🧰 **Utility Method: Create UI Canvas**

CreateUICanvas
This method ensures your scene has the **basic UI infrastructure**.

What it creates:

🎨 **Canvas object**

- Root for all UI
- Configured to render on screen
- Assigned to UI layer

📐 **UI Components added:**

- Scaler (handles resolution scaling)
- Raycaster (handles UI interaction)

🖱️ **Event System object**

- Required for UI interaction (clicks, navigation)

🔄 **Undo support**
Both objects are registered so:
👉 User can undo their creation

When is this used?
Only when:

- You are creating a UI prefab
- AND no Canvas exists in the scene

📦 **Core Method: Create and Configure Prefab**
CreateAndConfigurePrefab
This is the **main execution method** behind all menu actions.

Inputs:

- **fileName**
  - Name of the prefab to create
- **selectedGameObject**
  - What the user currently selected in the hierarchy
- **isUI**
  - Whether this prefab is UI (affects parenting and canvas creation)

Step-by-step behavior:

1. Handle UI requirement
If it's a UI prefab:

- Try to find an existing Canvas
- If none exists → create one

2. Find the prefab asset
Calls a helper method to locate the prefab in the project.
If not found:
❌ Logs error and stops execution

3. Determine parent
The prefab is placed under:

- Selected object (if something is selected)
- Otherwise:
  - Canvas (if UI)
  - Root of scene (if not UI)

4. Instantiate prefab
Creates a new instance of the prefab in the scene.

5. Finalize setup
Calls another method to:

- Register undo
- Unpack prefab
- Select it
- Enable renaming

🔍 **Utility Method: Find Prefab**
FindPrefabByName
This method searches your project for a prefab safely.

How it works:

1. Searches for prefab assets matching the name
2. Converts internal IDs into file paths
3. Normalizes path formatting
4. Ensures prefab is inside a specific folder:

👉 "Input System Extension/Prefab"

Why restrict the folder?
Prevents:

- Picking wrong prefab with same name
- Accidental conflicts with other assets

Matching logic:

- Compares file name (ignoring case)
- Returns the correct prefab if found

If not found:
❌ Logs error with clear message

🧊 **Finalization Method**
FinalizePrefabSetup
This handles **post-instantiation behavior**.

What it does:

1. Safety check

- If object is null → stop

2. Register Undo

- So creation can be undone

3. Unpack prefab
This is important:
👉 It removes prefab link
👉 Makes the object fully editable

Why unpack?
Without unpacking:

- Changes would affect prefab instance rules
- Harder to customize per scene

4. Select object

- Makes it the active selection in Unity

5. Trigger rename mode
Simulates pressing:
👉 **F2 key**

This allows:

- Immediate renaming after creation

Why delay?
Unity needs one frame to:

- Register selection properly
- Then accept input

📋 **Menu Items (User Entry Points)**
These are the **actual buttons you click in Unity**.

Location in Unity:
👉 GameObject → Tools → Input System Extension → UI

Available options:
Legacy UI

- Rebind Control Manager (Legacy)
- Button Reset All (Legacy)

TMP (TextMeshPro) UI

- Rebind Control Manager (TMP)
- Button Reset All (TMP)

General

- Input Display Manager

What each menu item does
Each one simply calls:
👉 Create prefab with:

- Specific name
- Current selection
- UI mode enabled

📦 **Variables involved (conceptually)**

canvas

- Stores UI root if needed

prefab

- The asset found in the project

parent

- Where the new object will be placed

instance

- The created object in the scene

🧪 **Usage (how you use it)**

Step-by-step:
1. Open Unity

2. Go to menu:
👉 **GameObject → Tools → Input System Extension → UI**

3. Choose an option
Example:
👉 "Rebind Control Manager (TMP)"

4. Result:

- Prefab appears in scene
- Canvas is created if needed
- Object is selected
- Rename mode starts instantly

💡 **Important behaviors**

Smart parenting

- If something is selected → becomes parent
- Otherwise → goes to Canvas or root

Automatic setup

No need to:

- Create Canvas manually
- Add EventSystem
- Drag prefabs manually

Safe asset lookup

Only uses prefabs from:

👉 Input System Extension folder

Fully editable objects

Because prefabs are unpacked:

- You can modify freely per scene

🧭 **How it fits your system**

Now you have a full pipeline:

1. Data System

Stores bindings and icons

2. Editor Tools

Generate mappings automatically

3. Settings Panel

Centralized configuration

4. Prefab Creator (this script)

👉 Instantiates ready-to-use UI

🚀 **Final insight**

This script is about **speed and usability**.

Without it:

❌ Drag prefab manually

❌ Create Canvas manually

❌ Fix hierarchy manually

Input System Extension Data

This script is the **central data container** of your entire system.

Everything else you built (debug logger, display manager, binding tools) depends on this.

Think of it as a **database + configuration file** that lives inside Unity.

🧠 **Big Picture — What this script does**

It stores:

🎮 **Input configuration**

- Which input asset is used
- Where bindings are saved

🎨 **Visual representation**

- Keyboard key → icon mapping
- Gamepad buttons → icons (Xbox / PlayStation)
- Default fallback icon

📦 **MAIN STRUCTURE**

The script is divided into 3 main parts:

1. **Input Settings**
2. **Icon Settings**
3. **Data Structures (mappings)**

🔧 **VARIABLES EXPLAINED**

🟢 **INPUT SETTINGS**

🔷 **defaultInputAction**

👉 What it is:

- A reference to your main input configuration (the action asset)

👉 What it does:

- Defines all actions like:
  - Jump
  - Move
  - Interact

👉 Why it matters:

- Other systems use this to:
  - read bindings
  - detect inputs
  - display correct icons

🔷 **playerPrefsKey**

👉 What it is:

- A string used as a **save key**

👉 What it does:

- Stores custom bindings as JSON in persistent storage

👉 Example:

- If player remaps controls → saved using this key

👉 Why it matters:

- Ensures bindings persist between sessions

🟡 **ICON SETTINGS**

🔷 **defaultSprite**

👉 What it is:

- A fallback icon

👉 Used when:

- No matching icon is found
- Mapping fails
- Data is missing

👉 Why it matters:

- Prevents UI from breaking or showing nothing

🔷 **KeyCodes**

👉 What it is:

- A list of keyboard mappings

Each entry contains:

- a key (like Space, A, Enter)
- a path name (like <Keyboard>/space)
- a sprite (image)

👉 What it does:
- Converts keyboard input into UI icons

👉 Example:
- Press "Space" → show spacebar icon

🔷 **xbox**

👉 What it is:
- A collection of Xbox-style controller icons

👉 Includes:
- A, B, X, Y
- triggers
- bumpers
- sticks
- D-pad

🔷 **ps**

👉 What it is:
- A collection of PlayStation-style icons

👉 Includes:
- Cross, Circle, Square, Triangle
- same structure as Xbox

📦 **STRUCTS (DATA TYPES)**

🧩 **InputSpriteList**

👉 **Purpose:**
Represents a **single keyboard mapping**

🔷 **Variables inside:**

keyCode
- Legacy input system key
- Used for compatibility/debugging

---

keyName
- New Input System path
- Example:
  - <Keyboard>/space
  - <Keyboard>/a

---

sprite
- The image shown in UI

🧠 **Why both keyCode and keyName?**

Because your system supports:
- old input system (KeyCode)
- new input system (path-based)

This allows **cross-compatibility and debugging**.

🎮 **GamepadIcons**

👉 **Purpose:**
Stores all sprites for a controller

🔷 **Categories inside:**

Face buttons

- South → A / Cross
- North → Y / Triangle
- East → B / Circle
- West → X / Square

Menu buttons
- Start
- Select

Triggers & bumpers
- LT / RT
- LB / RB

D-pad
- Full D-pad
- Up / Down / Left / Right

Analog sticks
- Left stick
- Right stick
- Press actions

🔧 **METHODS EXPLAINED**

🟢 **GetSprite(controlPath)**
👉 **Purpose:**
Given a **control name**, return the correct icon

🔍 **How it works:**
Input:
"buttonSouth"
Output:
returns sprite for A / Cross

📌 **Examples:**

Input Path	Result
buttonSouth	A / Cross
buttonEast	B / Circle
dpad/up	D-pad up icon
leftTrigger	LT / L2 icon

🧠 **Why this is important:**
This is how your UI system translates:
<Gamepad>/buttonSouth
into:
[A button icon]

🔴 **HandleUnknown(path)**
👉 **Purpose:**
Fallback when a mapping doesn't exist

What it does:
- Logs a warning
- Returns nothing

Why it matters:
Helps you detect:

- missing icons
- wrong bindings
- typos in paths

🔄 HOW THIS SCRIPT IS USED BY YOUR SYSTEM

🎯 InputDisplayManager
Uses this script to:
- get correct icons
- switch between keyboard/gamepad visuals
- handle PS vs Xbox differences

🧪 InputDebugLogger
Uses this to:
- check if keys exist in the database
- verify if sprites are assigned

🎮 Gameplay systems
Use it to:
- load default input actions
- save/load player bindings

🧠 FULL DATA FLOW
Example:
1. Player presses "Jump"
2. Input system detects action
3. Display system asks:

What icon should I show?

4. This asset answers:

- If keyboard → search KeyCodes
- If gamepad → use xbox/ps mappings

5. UI shows correct icon

⚙️ PRACTICAL USAGE

🧪 Setup in Unity
1. Create the asset (auto or menu)
2. Assign:
    - default input actions
    - keyboard sprites
    - gamepad sprites

🎨 Fill KeyCodes list
Example entries:

Key	Path	Sprite
Space	`<Keyboard>/space`	Space icon
A	`<Keyboard>/a`	A key icon

🎮 Assign gamepad icons
- Xbox icons → xbox section
- PlayStation icons → ps section

⚠️ IMPORTANT DESIGN INSIGHT
This script is **pure data**, not logic-heavy.
That's intentional.

Why this is good design:

✓ Keeps logic separate from data
✓ Easy to edit in inspector
✓ Reusable across systems
✓ Centralized configuration
✓ Easy to expand (new devices, new icons)

💡 **FINAL UNDERSTANDING**
This script is the **bridge between input and visuals**.
Without it:
- your system knows *what was pressed*
- but not *how to show it*

With it:
- inputs become **visual language**
- your UI becomes **dynamic and device-aware**

Input System Extension Data Inspector

This script is not part of gameplay itself — it's a **Unity Editor tool** that customizes how your configuration asset (InputSystemExtensionData) is edited and filled with data. Think of it as an **automation + UI helper inside the Unity Inspector**.
I'll break it down clearly: what it is, each part, and how you actually use it.

🧠 **What this script does (big picture)**
It replaces the default inspector of your configuration asset and adds a button:
👉 **"Get Default Icons"**
When pressed, it:
- Automatically fills **gamepad icon mappings** (Xbox + PlayStation)
- Automatically generates a **huge keyboard mapping list**
- Tries to find matching sprites in your project by name
- Falls back to a default sprite if something is missing

So instead of manually assigning hundreds of icons…
you press one button.

🧩 **Main Class: Custom Inspector**
InputSystemExtensionDataInspector
This is the core of the script.
It tells Unity:
"Whenever someone selects an InputSystemExtensionData, use THIS inspector instead of the default one."
It also supports:
- Editing **multiple assets at once**
- Running batch operations on all of them

🎨 **Method: Inspector Drawing**
OnInspectorGUI
This is responsible for **drawing what you see in the Inspector**.
What it does step-by-step:
1. **Syncs data**
   - Makes sure the editor is working with the latest values
2. **Draws a button**
   - Label: "Get Default Icons"
   - Tooltip explains what it does
3. **When clicked**
   - Loops through all selected assets
   - For each one:
     - Calls the generation method
     - Marks the asset as modified (so Unity saves it)
4. **Adds spacing**

    - Just visual organization
5. **Draws the normal inspector**
    - All your public fields still appear
6. **Applies changes**
    - Ensures edits are saved properly

⚙️ **Core Method: Data Generation**
GenerateKeyCodeList
This is the **heart of the system**.
It fills your data asset with everything it needs.

Step 1 — Reset existing data
- Clears the current keyboard mapping list
- Prevents duplicates or outdated entries

Step 2 — Define fallback sprite
- Uses defaultSprite
- This is used whenever a specific icon is not found

Step 3 — Generate Xbox icons
Creates a full set of mappings like:
- A button
- B button
- Triggers
- D-pad
- Sticks

Each one:
- Tries to find a sprite by name (like "XboxOne_A")
- Falls back if missing

Step 4 — Generate PlayStation icons
Same idea as Xbox, but using names like:
- "PS4_Cross"
- "PS4_Triangle"

This keeps platform-specific visuals consistent.

Step 5 — Generate keyboard mappings
This is the **largest part of the script**.
It builds a list where each entry contains:
- A key identifier (like "A", "Space", "F1")
- A control path (used by the Input System)
- A sprite (icon)

Examples of included keys:
- Letters (A–Z)
- Numbers (0–9)
- Function keys (F1–F15)
- Arrow keys
- Numpad keys
- Symbols (!, @, #, etc.)
- Modifier keys (Shift, Ctrl, Alt)
- Mouse buttons
- Joystick buttons (a LOT of them)

Important detail
Some entries have:
- Empty control paths
  This means:
  👉 They are only visual, not tied to input system paths

Final step

- Adds all generated entries into the asset

🔍 **Utility Method: Sprite Finder**

GetSprite

This is a helper that tries to find sprites automatically.

How it works:

1. Searches the entire project for sprite assets
2. For each asset:
    - Loads all sprites inside it
3. Compares names:
    - If a sprite name matches → returns it
4. If nothing matches:
    - Returns the default sprite

Important implication

Your sprite naming must match exactly:

Examples:

- "XboxOne_A"
- "PS4_Cross"
- "Space"
- "F1"

If names don't match → fallback is used.

📦 **Variables involved (conceptually)**

Even though this script doesn't declare many fields itself, it interacts heavily with:

From your data asset:

- **defaultSprite**
    - Fallback icon
- **KeyCodes**
    - List of keyboard mappings
- **xbox / ps**
    - Gamepad icon sets

🧪 **Usage (how you actually use this)**

Step-by-step in Unity:

1. Create or select an InputSystemExtensionData asset
2. In the Inspector:
    - You'll see the custom UI
3. Click:
   👉 **"Get Default Icons"**
4. The script will:
    - Fill all mappings automatically
    - Assign sprites if found

⚠️ **Requirements for it to work well**

1. Sprite naming must match

If your assets are named differently:

- It won't find them
- It will use the fallback

2. Sprites must be imported correctly

They need to be:

- Marked as **Sprite**
- Not just textures

3. Large project warning

This method scans:

👉 ALL sprites in your project

So:

- First run can be slow in big projects

💡 **Design insight**

This script is basically:

👉 A **data auto-population tool**

It solves a real problem:

- Manually assigning hundreds of icons is painful
- Error-prone
- Time-consuming

Instead:

- You define naming conventions
- The system does the rest

🧭 **How it fits into your system**

Together with your other scripts:

1. **Auto Creator**
   - Ensures the asset exists
2. **Data Asset**
   - Stores mappings
3. **Inspector (this script)**
   - Fills mappings automatically

👉 It's a clean pipeline:

- Create → Populate → Use at runtime

🚀 **Final takeaway**

This script is all about **automation and scalability**:

- One button replaces hours of manual setup
- Supports multiple assets
- Keeps your UI consistent across devices
- Enforces naming conventions

Input System Extension Data Window

This script builds a **Project Settings panel inside Unity** so you can manage your InputSystemExtensionData in a centralized place instead of hunting for the asset in the Project window.

Think of it as:

👉 *A control panel for your input system configuration.*

🧠 **What this script does (big picture)**

It adds a new section in:

Project Settings → Input System Package → Extension

Inside that panel, you can:

- Create the configuration asset if it doesn't exist
- View and access the asset quickly
- Auto-generate icon mappings
- Edit all settings directly (like a normal inspector)

🧩 **Main Component**

InputSystemExtensionDataWindow

This is a static utility responsible for **registering a custom settings page** in Unity.

It doesn't run in gameplay — only inside the editor.

⚙️ **Method: Creating the Settings Page**

CreateProvider

This method tells Unity:

“Add a new page in Project Settings with this layout and behavior.”

What it defines:

📍 Path

- "Project/Input System Package/Extension"

This determines where the settings appear in Unity’s Project Settings menu.

🏷️ Label

- "Extension"

This is the visible name in the UI.

🔍 Keywords

- "Input", "Bindings", "Gamepad", "Icons", "Controls"

These allow users to find the panel using Unity’s search bar.

🎨 GUI Handler

This is the **most important part**.
It defines **everything drawn in the settings panel UI**.

🎨 GUI Flow (step-by-step)

The interface is built dynamically every time the panel is opened.

1. Load the data asset

It tries to retrieve your configuration:

- If found → show full UI
- If missing → show creation UI

2. Info message

Displays a help box explaining what the system is for:

👉 Input bindings + icon mappings configuration

⚠️ Case 1: Asset does NOT exist

If the configuration asset is missing:
What the UI shows:

- Warning message
- Button: **“Create Settings Asset”**

What happens when clicked:

1. Calls the auto-creator system
2. Creates (or finds) the asset
3. Highlights it in the Project window
4. Selects it
5. Forces UI refresh

Why the refresh is needed

Unity’s UI system can break layout if you modify structure mid-draw, so it safely exits and redraws everything.

✅ Case 2: Asset exists

Now the full editor UI becomes available.

📦 Section: Data Asset

Displays the current configuration asset.
Elements:

- **Read-only field**
  - Shows the asset reference
- **Ping button**
  - Highlights the asset in the Project window

- **Open button**
  - Opens the asset in the Inspector

🛠 **Section: Tools**
Button: "Get Default Icons"
This is the same functionality from your custom inspector.

What it does:
1. Registers undo (so user can revert)
2. Generates:
   - Keyboard mappings
   - Gamepad icons
3. Marks asset as modified
4. Saves it immediately

Why Undo is important
Without it:
- Changes would be permanent instantly
- User couldn't revert mistakes

📜 **Section: Full Inspector Drawing**
This part replicates the normal inspector — but inside Project Settings.

How it works:
1. Creates a serialized version of the asset
   (this is a safe editable representation)
2. Iterates through all properties
3. For each property:
   - Draws it in the UI
   - Supports nested structures
4. Skips internal script reference field
   (Unity handles that automatically)

✏️ **Applying Changes**
After editing:
1. Registers undo again
2. Applies changes back to the real asset
3. Marks it as dirty
4. Saves it to disk

📦 **Variables involved (conceptually)**
This script doesn't store much itself — it works on the data asset.
But internally it uses:

data
- The main configuration asset
- Contains all input + icon settings

so (Serialized Object)
- Editable wrapper around the asset
- Tracks changes safely

prop
- Iterator used to walk through all fields
- Allows dynamic UI drawing

🧪 **Usage (how you use this in practice)**

Step-by-step

1. Open Project Settings

Go to:

👉 **Edit → Project Settings**

2. Navigate to your panel

Find:

👉 **Input System Package → Extension**

3. If asset is missing

Click:

👉 **Create Settings Asset**

4. Configure or generate data

- Press **Get Default Icons** to auto-fill everything
- Or manually tweak values

5. Done

Everything is saved automatically and ready for runtime use.

💡 **Why this script is useful**

Without it:

❌ You must manually find the asset
❌ No centralized config
❌ Harder for teams to use

With it:

✅ Centralized configuration
✅ Easy access from Project Settings
✅ One-click setup
✅ Undo support
✅ Cleaner workflow

🧭 **How it fits into your system**

This script is the **final layer of usability**:

1. Auto Creator

Ensures the asset exists

2. Data Asset

Stores everything

3. Custom Inspector

Adds automation tools

4. Project Settings (this script)

👉 Makes everything accessible and user-friendly

🚀 **Final insight**

This is a **tooling-quality upgrade**, not a gameplay feature.

You've essentially built:

👉 A mini "package settings panel" like Unity's own systems

That's exactly how professional Unity tools behave.

Input System Extension Data Auto Creator

This script is part of your **editor-side infrastructure**. Its job is simple but very important:
it guarantees that a central configuration asset (your input data database) **always exists**, without you needing to remember to create it manually.

🧠 **Big Picture — What this script does**
Think of this as an **auto-installer + safety net** for your system.
It ensures that a file called:
Input System Extension Data

- exists in your project,
- is placed in the correct folder (Resources),
- and is created automatically if missing.

It works in **two ways**:

1. Automatically when the editor starts.
2. Manually via a menu option.

🧩 **Main Components**
There are **two main parts**:
1. InputSystemExtensionDataAutoCreator
Handles creation, finding, and replacement of the asset.
2. InputSystemExtensionDataStartup
Triggers automatic creation when Unity starts.

📦 **VARIABLES & STRUCTURE**
This script doesn't have many stored variables—it's mostly **logic-driven**—but there are important **temporary values** inside methods.

🔧 **METHOD BREAKDOWN**

🟢 **CreateOrGetData(allowOverwritePrompt)**
👉 **Purpose:**
This is the **core method**.
It either:

- finds an existing asset, or
- creates a new one.

🔍 **Step-by-step logic:**
1. Find base folder
It searches for a folder named:
Input System Extension

- If found → use it.
- If not → fallback to:

Assets

2. Ensure a Resources folder exists
Why?
Because assets inside Resources can be loaded at runtime.
If it doesn't exist:

- it creates it automatically.

3. Define asset path
It builds a path like:
[BaseFolder]/Resources/Input System Extension Data.asset

4. Try loading existing asset
If the asset already exists:

- If overwrite is **not allowed** → return it immediately.
- If overwrite **is allowed**:
  - Show a confirmation dialog.

- If user says "No" → keep existing.
- If "Yes" → delete and recreate.

5. Create new asset
If no asset exists (or user chose to overwrite):
- Create a new data object.
- Save it into the project.
- Mark it as "dirty" (so Unity saves it).
- Refresh the asset database.

🧠 **Key Variables inside this method:**
- baseFolderPath → where your system folder is.
- resourcesPath → ensures runtime accessibility.
- assetPath → final file location.
- existingAsset → already created asset (if any).
- asset → newly created asset.

🟡 **CreateCustomObjectData()**
👉 **Purpose:**
Manual creation via Unity menu.

What it does:
- Calls CreateOrGetData(true)
  → allows overwrite prompt.
- Focuses the project window.
- Selects the created asset.

🧪 **Usage:**
You'll find this in Unity under:
Assets → Create → Tools → Input System Extension → Input System Extension Data

🔵 **FindInputSystemExtensionFolder()**
👉 **Purpose:**
Searches your project for the correct folder.

How it works:
- Scans all folders in the project.
- Looks for exact name:

Input System Extension
- If found → returns its path.
- If not → returns null.

🧠 **Why this matters:**
This allows your system to:
- work even if moved around in the project,
- stay flexible and portable.

🟣 **InputSystemExtensionDataStartup**
This is the **automatic trigger system**.

🟢 **Static Constructor**
Runs **automatically when Unity loads**.

What it does:
- Waits until Unity is fully initialized.
- Calls:

CreateOrGetData(false)
Meaning:

- It creates the asset if missing.
- It does NOT ask before overwriting (safe mode).

🔄 **FULL FLOW (Simplified)**

When Unity starts:

1. Script runs automatically.
2. Checks if asset exists.
3. If missing → creates it silently.

When you use the menu:

1. You click the menu option.
2. It checks if asset exists.
3. If yes → asks if you want to replace.
4. Creates or selects the asset.

🧠 **WHY THIS SCRIPT IS IMPORTANT**

Without this:

- Your system could break if the asset is missing.
- You'd rely on manual setup (error-prone).
- Other scripts depending on this data could fail.

⚙️ **HOW IT FITS INTO YOUR SYSTEM**

This script supports:

- your **InputDisplayManager**
- your **KeyCode ↔ sprite mapping system**
- your **input visualization system**

It guarantees that:

✓ sprite mappings exist

✓ input data is always accessible

✓ runtime systems won't crash due to missing data

🧪 **PRACTICAL USAGE**

You don't need to do anything most of the time.

Just:

- open Unity → asset is ensured automatically.

If you want to regenerate:

Use the menu:

Assets → Create → Tools → Input System Extension → Input System Extension Data

⚠️ **DESIGN NOTES (Important Insight)**

This script is **defensive programming**:

- It assumes users might forget setup.
- It fixes problems before they happen.
- It removes friction from your workflow.

💡 **Final Insight**

This is not just an asset creator — it's a **guarantee system**.

It ensures your entire input extension framework always has its **foundation ready**, making all other systems (debug, display, bindings) reliable.

On Input System Event

This script is one of the most advanced pieces in your system. It acts like a **central event dispatcher for input**, turning raw input values into meaningful events like **Pressed**, **Holding**, and **Released**—while also allowing multiple systems to safely share the same input.

I'll break it down in a clear way: **what it is**, then **each variable**, then **each method**, and finally **how to use it in practice**.

🧠 **Core Idea (What this script does)**
Instead of checking input manually everywhere, this system:

- Watches an input (like movement, jump, trigger, etc.)
- Detects **state transitions**:
  - Pressed → first frame input becomes active
  - Hold → input stays active
  - Released → input stops
- Lets **multiple scripts listen to the same input safely**
- Keeps each listener independent using an **owner system**

👉 Think of it like a **signal hub**:
"When this input changes, notify everyone who registered for it."

📦 **Main Static Class**
OnInputSystemEvent<T>
This is the **global manager**.

- It works with any input type:
  - number (trigger)
  - 2D direction (movement)
  - boolean (button)

🔷 **Variable: states**
This is the **core storage of everything**.

- It links:
  - one input → one internal state object
- Each state contains:
  - callbacks
  - last value
  - per-owner tracking

👉 Think of it as:
"All active input listeners grouped by input."

🧩 **Internal State Class**
State
This represents **one input being tracked**.

🔷 **Variable: action**

- The actual input being monitored.

🔷 **Variable: lastValue**

- Stores the previous frame value.
- Used to detect changes.

🔷 **Lists of callbacks**
Each list stores callbacks grouped by **owner**:

- OnHold
- OnPressed
- OnReleased

Each entry contains:

- owner (who registered)
- callback (what to execute)
- enabled condition (when it should run)

👉 This allows:
Multiple systems using the same input without conflict.

🔷 **Variable: ownerWasHeld**

- Tracks if each owner was previously holding the input.
- This is critical for detecting:

- Press transitions
- Release transitions

🔁 **Core Method: Update**
This is the **heart of the system**.
Runs every frame after input updates.

Step-by-step logic:
1. Validate input
- If input is missing or disabled → stop

2. Read current value
- Gets current input value (example: joystick direction)

3. Determine if input is "active"
- Uses a helper function to decide:
  - Is this value meaningful?
  - Example:
    - small joystick movement = ignored
    - real movement = active

4. Collect all owners
- Gathers everyone listening to this input

5. Process each owner independently
For each owner:
Check if owner is enabled
- Each owner can decide if it wants to listen right now

CASE A — Owner disabled
If:
- owner was holding before → trigger **Release**

Then skip everything else.

CASE B — Input is active
If input is active:
- If it was NOT active before:
  → trigger **Pressed**
- Always:
  → trigger **Hold**

CASE C — Input became inactive
If it was active before:
→ trigger **Released**

6. Save current value
- Used for next frame comparisons

🔍 **Helper Methods inside State**

GetEnabledForOwner
- Finds the "enabled condition" for an owner
- This lets each system decide:
  - "Should I react right now?"

IsNonZero
This determines if input is "active".
Examples:
- float → must be above small threshold

- vector → must have magnitude
- bool → true = active

👉 Prevents noise and false triggers.

UnregisterAll
- Removes all callbacks from an owner
- Clears its internal state

📡 **Registration Methods (Public API)**
These are how external systems connect to the dispatcher.

RegisterHold
- Called every frame while input is active

RegisterPressed
- Called once when input starts

RegisterReleased
- Called once when input stops

UnregisterAll
- Removes all callbacks from a specific owner

Clear
- Completely removes an input from the system

⚙️ **Configuration Builder**
WithAction
This is a **fluent entry point** (clean setup style).
What it does:
- Ensures input is enabled
- Gets or creates internal state
- Returns a configuration object

GetOrCreateState
- If input is new → create a state
- If already exists → reuse it

Also:
- Hooks into the engine update loop
- Ensures Update runs every frame

🧩 **Configuration Class**
OnInputSystemEventConfig<T>
This is a **helper to chain setup calls**.

🔷 **Variables**
- action → input being used
- owner → who owns the callbacks
- enabled → condition to allow execution

🔷 **Methods**

OnHold
- Registers hold behavior

OnPressed
- Registers press behavior

OnReleased

- Registers release behavior

Dispose

- Removes all callbacks for this owner

🎮 **Usage Example (Conceptual)**

Imagine:

- Player movement system
- UI navigation system
- Both use the same joystick

Without this system:

❌ They would conflict

With this system:

Each system registers separately with its own owner.

Example flow:

1. A script registers:
    - Press → jump
    - Hold → charge jump
    - Release → execute jump
2. Another script registers:
    - Same input → UI navigation
3. Each has:
    - its own enabled condition

👉 Result:

- Both systems coexist
- No interference
- Clean separation

🧠 **Key Strengths of This System**

✅ **Multi-owner safe**

Multiple systems can use same input.

✅ **State-based logic**

Automatically detects:

- press
- hold
- release

✅ **Flexible enable control**

Each owner decides when it is active.

✅ **Generic**

Works with:

- buttons
- sticks
- triggers
- complex values

✅ **Centralized logic**

No need to write input checks everywhere.

⚠️ **Important Notes**

- You must unregister (or dispose) when object is destroyed
- Each owner should be unique (usually the object itself)
- Avoid creating too many duplicate registrations

📜 **Summary**
This script is essentially:
A **shared input event system** that transforms raw input into structured events and distributes them safely to multiple listeners.
It replaces:

- scattered input checks
- duplicated logic
- conflicting systems

with:

- a clean, centralized, scalable solution

Input System Utility

This script is a **general-purpose input detector**.
Its job is simple but very useful:
👉 Detect **any kind of player interaction**, regardless of the device being used.

🧠 **Core Idea**
Instead of checking input like this everywhere:

- "Did the keyboard do something?"
- "Did the mouse click?"
- "Did the controller press a button?"

This script centralizes everything into **clean, reusable checks**.

🧱 **Structure Overview**

- It is a **static utility** → no need to attach it to anything.
- All methods are **global helpers**.
- Focused on **"input happened this frame" detection**.

⚙️ **Methods Explained**

🔷 **IsAnyInputPressedThisFrame()**
Purpose:
Detects if **any input from any device** was pressed in the current frame.

How it works:
It combines all other methods:

- Keyboard
- Mouse
- Gamepad
- Touch

If **any one of them returns true** → this method returns true.

Think of it as:
👉 "Did the player interact with *anything* right now?"

Usage examples:

- Press any key to continue screens
- Idle detection systems
- Wake-up UI (e.g., "Press any button")

🔷 **IsKeyboardPressed()**
Purpose:
Checks if **any keyboard key** was pressed this frame.

How it works:

1. Verifies that a keyboard exists.

2. Checks if **any key** was pressed this frame.

Important detail:
- It detects only **new presses**, not keys being held.

Usage examples:
- Detect typing start
- Keyboard-only prompts
- Switching input mode (keyboard vs controller)

🔷 **IsMousePressed()**
Purpose:
Detects mouse button clicks.

How it works:
1. Verifies that a mouse exists.
2. Checks the main buttons:
    - Left
    - Right
    - Middle

If any of them were pressed → returns true.

What it does NOT detect:
- Movement
- Scroll wheel

👉 Only button presses.

Usage examples:
- Detect clicks for UI activation
- Switch to mouse input mode
- Pause idle timers

🔷 **IsGamepadPressed()**
Purpose:
Detects **any button press on any connected controller**.

How it works:
1. Loops through all connected controllers.
2. For each controller:
    - Loops through all its controls (buttons, triggers, etc.).
3. Checks:
    - If the control is a button.
    - If it was pressed this frame.

If any match → returns true.

Important detail:
👉 This is very flexible:
- Works with Xbox, PlayStation, generic controllers, etc.
- Doesn't depend on specific button names.

Trade-off:
- Slightly more expensive than other checks (loops through controls).
- But very powerful and generic.

Usage examples:
- Detect controller activity
- Switch UI hints (keyboard → gamepad icons)
- Wake up UI from controller input

🔷 **IsTouchPressed()**
Purpose:
Detects touch input (for mobile devices).

How it works:
1. Verifies a touchscreen exists.
2. Checks if the **primary touch** was pressed this frame.

Important detail:
- Focuses only on the main touch.
- Does not track multiple touches.

Usage examples:
- Mobile UI interaction detection
- Tap-to-start screens
- Mobile idle detection

🔄 **Frame-Based Detection**
All methods use the concept of:
👉 **"Pressed this frame"**
This means:
- Returns true **only once**, at the moment of press.
- Not while holding the button.

Why this matters:
Prevents:
- Repeated triggers
- Input spam
- Accidental multiple activations

🎮 **Usage Scenarios**

🟢 **1. "Press Any Button to Start"**
You only need:
- Call the global method
- If true → continue game

🟢 **2. Input Device Detection**
Track last input type:
- If keyboard used → show keyboard UI
- If gamepad used → show controller icons

🟢 **3. Idle System**
Reset inactivity timer when:
- Any input is detected

🟢 **4. UI Wake-Up**
- Hide UI after inactivity
- Show again when input is detected

💡 **Design Strengths**
✓ **Centralized logic**
All input detection in one place.

✓ **Device-agnostic**
Works across:
- Keyboard
- Mouse

- Gamepad
- Touch

✓ **Easy to use**
One-line checks for complex behavior.

✓ **Expandable**
You can easily add:
- VR input
- Specific device filters
- Axis movement detection

⚠ **Limitations**
❗ **Gamepad detection cost**
- Iterates through all controls.
- Fine for most games, but not ideal for tight performance loops.

❗ **Mouse limitations**
- Does not detect movement or scrolling.

❗ **Touch simplification**
- Only checks primary touch.
- No multi-touch support.

🧩 **Summary**
This script is a **universal input detector**:
- Checks multiple device types
- Detects only **new input events**
- Provides clean and reusable methods
- Helps unify input handling across platforms

Input System Extension Helper

This script is a **small but critical utility** in your system.
It acts as a **central access point** for your configuration data — specifically the InputSystemExtensionData.

🧠 **Core Idea**
Instead of loading your configuration asset everywhere in your project:
👉 This script loads it **once** and then **reuses it everywhere**.

📦 **Variable Explained**
🔷 **extensionData**
- Type: reference to your configuration asset.
- Scope: static (shared globally).
- Purpose: **cache (store in memory)** the loaded data.

Why this exists
Loading from the Resources folder is relatively expensive if done repeatedly.
So instead of doing:
- Load → use → discard → load again...

This script does:
- Load once → store → reuse forever

👉 This improves performance and keeps things clean.

⚙ **Method Explained**
🔷 **GetInputSystemExtensionData()**

This is the **only public method** in the script.
Purpose:
Provides safe and efficient access to your configuration data.

🔄 **Step-by-step behavior**
1. Check if already loaded

- If extensionData already has a value:
  - It immediately returns it.
  - No loading happens again.

👉 This is the **cache system**.

2. Load from Resources (only if needed)
If the data is not loaded yet:

- It tries to load it from the **Resources folder**.
- It looks for a file with a **specific name**:

👉 "Input System Extension Data"
⚠️ Important:

- The name must match exactly.
- The file must be inside a Resources folder.

3. Validate result
If loading fails:

- It logs an error.
- Returns null.

👉 This helps detect setup issues early.

4. Return the data
If successful:

- Returns the loaded asset.
- Keeps it cached for future calls.

🔄 **Lifecycle Behavior**
First time something calls this method:

- Asset is loaded from disk.
- Stored in memory.

Every next time:

- Instantly returned from memory.

🎮 **Usage in Your System**
This helper is used by many other systems, for example:

🔷 **Rebinding system**

- Needs icon mappings
- Needs input settings

👉 Calls this method to get them.

🔷 **Save system**

- Needs the input asset
- Needs the PlayerPrefs key

👉 Also retrieves it through this helper.

🔷 **UI systems**

- Need access to configuration data

👉 Same method.

🧩 **Why this design is good**
✓ **Centralized access**

- One place controls how the data is loaded.
- No duplication across scripts.

✓ **Performance optimized**
- Loads only once.
- Avoids repeated disk access.

✓ **Safe usage**
- Built-in error checking.
- Prevents silent failures.

✓ **Easy to maintain**
- If loading logic changes, update it in one place only.

⚠ **Important Requirements**
For this to work correctly:
1. File location
- Must be inside a **Resources folder**.

2. File name
- Must be exactly:

Input System Extension Data
(no extension in the name used for loading)

3. Only one asset expected
- If multiple exist, behavior may become unpredictable.

🧠 **Mental Model**
Think of this script like a **singleton data provider**:
- First call → fetches the data.
- Future calls → reuses it.

🧩 **Summary**
This script:
- Loads your configuration asset from disk.
- Stores it in memory.
- Provides global access to it.
- Prevents redundant loading.
- Ensures all systems use the same data instance.

Input Binding Saver

This script is responsible for one very important part of any rebinding system:
👉 **Persisting (saving) and restoring (loading) the player's custom input bindings.**
Without this, every time the player closes the game, all their custom controls would be lost.

🧠 **Core Idea**
- When the player changes controls → **save the changes**.
- When the game starts → **load those changes back**.

It uses Unity's built-in storage system (**PlayerPrefs**) and converts bindings into a **JSON string**.

📦 **Structure Overview**
This is a **static utility**, meaning:
- It does not exist as a component in the scene.
- You don't attach it to anything.
- You call it from other systems (like your rebind manager).

📦 **Variables (Implicit Dependencies)**

This script doesn't define its own fields, but it relies on:

🔷 **extensionData**

Retrieved through a helper function.

It contains:

- The **input action asset** (all controls).
- The **PlayerPrefs key** (where data is stored).

🔷 **InputActionAsset**

This is:

👉 The full collection of input actions and bindings.

It includes:

- Actions (Jump, Shoot, Move, etc.)
- Their bindings (keys, buttons, etc.)
- Any overrides created at runtime

🔷 **playerPrefsKey**

A string used as an identifier.

👉 Think of it like a "save slot name":

- Used to store and retrieve the JSON data.

⚙ **Methods Explained**

🔷 **SaveDefaultBindings()**

Purpose:

Saves the current bindings using the default configuration.

What it does:

1. Gets the central data object.
2. Checks if it exists.
3. Calls the generic save method using:
    - The default input actions.
    - The predefined PlayerPrefs key.

When it is used:

- After a player finishes rebinding controls.
- After resetting bindings.
- Anytime you want to persist changes.

🔷 **LoadDefaultBindings()**

Purpose:

Loads saved bindings automatically when the game starts.

Special behavior:

👉 It runs **automatically on game startup**.

What it does:

1. Gets the central data object.
2. Checks if it exists.
3. Calls the generic load method using:
    - The default input actions.
    - The predefined PlayerPrefs key.

Why this is important:

- Ensures player preferences are restored every time the game launches.
- No manual call needed.

🔷 **SaveBindings(inputActionAsset, playerPrefsKey)**

Purpose:

Generic method to save any set of bindings.

Step-by-step:

1. **Validation**
    - If the input system asset is missing → stop.

2. **Convert bindings to JSON**
    - All current binding overrides are serialized into a string.

👉 This includes:

- Only the changes made by the player (not defaults).

3. **Store in PlayerPrefs**
    - Saves the JSON string using the provided key.

4. **Force save**
    - Writes data to disk immediately.

5. **Debug log**
    - Confirms the save operation.

Key concept:

👉 The system does NOT save the entire input setup

👉 Only the **overrides** (what changed)

🔷 **LoadBindings(inputActionAsset, playerPrefsKey)**

Purpose:

Restores saved bindings and applies them.

Step-by-step:

1. **Validation**
    - If the input system asset is missing → stop.

2. **Check if data exists**
    - If no saved key is found → do nothing.

👉 This avoids errors on first launch.

3. **Retrieve JSON**
    - Reads the saved string from PlayerPrefs.

4. **Apply overrides**
    - Injects the saved bindings back into the input system.

5. **Debug log**
    - Confirms loading.

Important:

👉 This does NOT replace the base bindings

👉 It **overrides them dynamically**

🔄 **Full Lifecycle**

Here's how everything works together:

🎮 **First time player runs the game:**

- No saved data exists.
- Default bindings are used.

🎮 **Player changes controls:**

- Rebind system updates bindings.
- SaveDefaultBindings() is called.
- Overrides are saved as JSON.

🎮 **Player closes and reopens the game:**

- LoadDefaultBindings() runs automatically.
- JSON is loaded.
- Overrides are applied.
- Player sees their custom controls again.

💾 **Why JSON?**

Because bindings are complex:

- Multiple actions
- Multiple devices
- Composite bindings

JSON allows:

- Compact storage
- Easy serialization/deserialization
- Flexibility

🎮 **Usage Example**

Typical integration:

After a successful rebind:

- Call save.

After reset:

- Call save again.

On game startup:

- Load happens automatically.

💡 **Design Strengths**

✓ **Fully automatic loading**

No need to manually trigger loading.

✓ **Clean separation**

- UI handles interaction.
- This script handles persistence.

✓ **Flexible**

You can reuse SaveBindings and LoadBindings for:

- Different profiles
- Different control schemes
- Multiple save slots

⚠️ **Limitations**

❗ **Uses PlayerPrefs**

- Not ideal for large-scale save systems.
- Stored as plain text (not encrypted).

❗ **No versioning**

- If bindings structure changes, old saves may break.

❗ **Single key system**

- Only one saved configuration unless you extend it.

🩹 **Summary**

This script is the **bridge between runtime input changes and persistent storage**:

- Converts binding changes → JSON
- Saves them locally
- Loads them automatically on startup
- Applies them back into the input system

Rebind Control Manager and Rebind Control Manager TMP

This script is the **core of your rebinding system UI**. It connects the Input System with UI elements (TextMeshPro + Images + Buttons) and allows the player to **change controls at runtime**, while also handling edge cases like duplicates, composites, canceling, and saving.
I'll break it down clearly into:

1. **What the script does (big picture)**
2. **Each variable explained**
3. **Each method explained**
4. **How everything flows together (lifecycle)**
5. **How to use it in practice**

🧠 1. Big Picture

This component acts as a **controller for one rebindable input**.
It:

- Displays the current binding (text or icon).
- Lets the player click a button to rebind it.
- Shows a UI overlay while waiting for input.
- Prevents duplicate bindings.
- Supports composite bindings (like WASD).
- Allows canceling the rebind.
- Saves changes.
- Notifies other systems (like your icon handler).

📦 2. Variables Explained

🔷 Input Settings

cancelRebindActionReference

- A reference to an input action used to **cancel the rebinding process**.
- Example: pressing ESC or B cancels.

inputActionReference

- The **input action being modified**.
- Example: "Jump", "Move", "Shoot".

targetBindingId

- A unique identifier that tells **which specific binding inside the action** will be changed.
- Needed because one action can have multiple bindings.

displayOptions

- Controls how the binding is shown as text.
- Example:
  - Show device name
  - Show control path
  - Show simplified name

🔷 UI Elements

actionLabel

- Text showing the **name of the action** (e.g., "Jump").

autoLabelFromAction

- If enabled, the label automatically uses the action name.

startRebindButton

- Button the player presses to **start rebinding**.

bindingDisplayText

- Text showing the **current binding** (e.g., "Space", "A").

actionBindingIcon
- Image used to show an **icon instead of text**.

rebindOverlayUI
- A UI panel shown during rebinding.
- Usually blocks input and shows instructions.

rebindPromptText
- Text like:
  - <Press a key>
  - <Press W>

actionRebindIcon
- Icon shown during rebinding (preview of pressed input).

resetButton
- Button to restore default binding.

🔷 **Events**

onRebindStart
- Triggered when rebinding begins.

onRebindStop
- Triggered when rebinding ends (success or cancel).

onUpdateBindingUI
- Triggered whenever UI needs to refresh.
- Used by systems like your **icon replacer**.

🔷 **Private Fields**

currentRebind
- Stores the active rebinding operation.

allRebindManagers
- A global list of all instances.
- Used to update all UIs when something changes.

currentEventSystem
- Reference to the UI event system.
- Used to enable/disable UI interaction safely.

⚙️ **3. Methods Explained**

🔷 **Awake**
- Runs early.
- Checks if required references are missing.
- Logs warnings.

👉 Purpose: **Validation**

🔷 **Start**
- Hides the overlay.
- Connects button clicks to:
  - Start rebind
  - Reset binding
- Stores EventSystem.

👉 Purpose: **Initial setup**

🔷 **OnEnable**

- Adds this instance to the global list.
- Subscribes to input system changes (only once globally).

🔷 **OnDisable**
- Cleans up:
  - Cancels active rebinding
  - Removes from global list
  - Unsubscribes if last instance

🔷 **OnValidate (Editor only)**
- Keeps UI updated while editing in inspector.

🔁 **Core Logic Methods**

🔷 **TryResolveBinding**
- Finds:
  - The action
  - The index of the binding using the ID

👉 If it fails → stops everything.

🔷 **RefreshBindingDisplay**
- Updates UI:
  - Gets binding text
  - Detects if binding was changed
  - Updates reset button state
  - Fires update event

👉 This is the **main UI refresh method**.

🔷 **ResetToDefault**
- Removes custom overrides.
- Also resets composite parts.
- Refreshes UI.
- Saves bindings.

🔷 **StartInteractiveRebind**
- Entry point when user clicks the button.

Logic:
- If binding is composite → start from first part
- Otherwise → rebind directly

🔷 **ExecuteInteractiveRebind**

This is the **heart of the system**.

It:
1. Cancels any existing rebind.
2. Disables gameplay input.
3. Disables UI interaction.
4. Starts listening for input.

During the process:

➤ **OnCancel**
- Stops rebinding
- Restores UI
- Cleans up

➤ **OnComplete**
- Applies new binding
- Checks duplicates

- Saves
- Updates UI
- Moves to next composite part (if needed)

➤ **OnPotentialMatch**
- Runs BEFORE accepting input.
- Checks if input is already used elsewhere.
- Cancels if duplicate.

Also:
- Shows overlay UI
- Updates prompt text
- Starts cancel monitoring

🔷 **CheckDuplicateBindings**
- Searches entire input system.
- Prevents assigning same input twice.

🔷 **GenerateCustomPrompt**
- Builds text like:
  - <W>
  - <Up>/<Down>/<Left>/<Right>
- Highlights current part in composite bindings.

🔷 **MonitorCancelInput**
- Enables cancel action.
- Listens for cancel input.

🔷 **OnCancelInputPerformed**
- Cancels current rebind.

🔷 **HandleActionChange (Static)**
- Runs when input system changes globally.
- Updates all relevant managers.

👉 Keeps UI in sync automatically.

🔷 **UpdateActionLabel**
- Updates label text with action name.

🔄 **4. Full Flow (Step-by-Step)**

🟢 **When the game starts:**
- UI is initialized.
- Buttons are wired.
- Binding text is shown.

🟡 **When player clicks "Rebind":**
1. UI overlay appears
2. Input system waits for input
3. Gameplay input is disabled
4. UI interaction is disabled

🔵 **When player presses a key:**
- System checks:
  - Is it duplicate?
- If valid:
  - Applies binding
  - Saves

- Updates UI

🔴 **If player cancels:**
- Rebind stops
- UI restores

🟣 **If binding is composite:**
- Rebind continues through all parts

🎮 **5. How to Use It**

Step 1 — Add to UI Object
Attach this component to a GameObject.

Step 2 — Assign References
You must link:
- Input Action
- Binding ID
- UI elements (texts, buttons, images)

Step 3 — Setup Buttons
- Rebind button → starts rebinding
- Reset button → restores default

Step 4 — Optional Systems
You can connect:
- Icon system (like your other script)
- Save system
- Custom UI feedback

Step 5 — Play
- Click button
- Press new input
- Binding updates instantly

💡 **Key Strengths of This Script**
✓ Handles **real-time rebinding**
✓ Supports **composite inputs (WASD, sticks)**
✓ Prevents **duplicate bindings**
✓ Works with **UI + icons**
✓ Supports **cancel input**
✓ Auto-syncs across all instances
✓ Fully event-driven (extensible)

⚠️ **Important Notes**
- The system depends heavily on **binding IDs** → must be correct.
- UI references must be assigned → or you'll get warnings.
- Works best when paired with:
  - your icon handler script
  - a binding save/load system

Gamepad Icon Rebind Handler and Gamepad Icon Rebind Handler TMP

This script is a **runtime UI behavior**.
Its job is simple but powerful:
👉 **Replace input text (like "Press A") with icons (like the A button image)** depending on the controller being used.

🧠 **What this script does (big picture)**
It listens to updates from your rebind UI system and decides:

- If an icon exists → show the icon
- If not → show text

So your UI automatically adapts to:

- Xbox controllers
- PlayStation controllers
- Keyboard / unknown inputs

🧩 **Main Component**
GamepadIconRebindHandlerTMP
This is a component you attach to a GameObject in your scene.
It acts like a **bridge between**:

- Your input rebinding system
- Your icon database (ScriptableObject)
- Your UI elements (text + images)

📦 **Inspector Variable**
rebindControlGroup
This is the **only visible field in the inspector**.

What it represents:

- A parent object in your scene
- Contains multiple **rebind UI elements**

Why it exists:
Instead of manually linking each UI element:
👉 You give it one parent
👉 It automatically finds everything inside

🔒 **Private Variable**
extensionData
This stores your configuration data:

- Keyboard mappings
- Gamepad icon mappings

Where it comes from:
Loaded using your helper system at runtime.

Why it matters:
This is where the script gets:
👉 “Which icon should I use for this button?”

▶️ **Unity Method: Start**
Start
This runs automatically when the scene begins.

Step-by-step behavior:

1. Load configuration data

- Retrieves the ScriptableObject with all mappings
- Ensures it’s available before doing anything

2. Find all rebind managers
Inside the rebindControlGroup, it searches for:
👉 All UI components responsible for displaying bindings

3. Subscribe to updates

For each manager:

- It listens to an event that triggers whenever the UI updates
- Connects that event to this script's handler

4. Force initial refresh
Immediately tells each manager:
👉 "Update your display now"
So icons appear correctly from the start.

🔔 **Event Method**
OnBindingUIUpdate
This is the **core logic** of the script.
It runs every time a binding UI element updates.

Inputs it receives:

- **manager**
  - The UI element being updated
- **bindingDisplay**
  - The text version of the binding (like "A" or "Space")
- **deviceLayoutName**
  - What type of device is being used
  - Example: PlayStation, generic gamepad
- **controlPath**
  - Internal identifier of the input
  - Example: "buttonSouth", "dpad/left"

⚙️ **Step-by-step logic**

1. Validate inputs
If anything important is missing:

- Device type
- Control path
- Data asset

👉 The method stops immediately

2. Prepare icon variable
Starts with:
👉 No icon selected yet

3. Detect controller type
This is a key part.

Case 1: PlayStation controller
If device matches PlayStation layout:
👉 Use PlayStation icon set

Case 2: Generic gamepad (Xbox-style)
If device matches general gamepad:
👉 Use Xbox icon set

Important detail
Xbox icons act as:
👉 **default fallback for most controllers**

4. Get icon from mapping
Using the control path:

- Looks up the correct sprite from your data
- If found → assign it

- If not → remains empty

🎯 **UI References (from manager)**
The script then retrieves UI elements:

Text elements:
- Main binding text
- Rebinding prompt text

Image elements:
- Main icon image
- Rebinding icon image

Safety checks
If any reference is missing:
❌ Logs an error
❌ Stops execution

🔄 **Final Decision: Icon vs Text**

Case 1: Icon exists
👉 Show icons, hide text
- Assign sprite to both icon images
- Enable icon objects
- Disable text objects

Case 2: Icon does NOT exist
👉 Show text, hide icons
- Disable icon objects
- Enable text objects

💡 **Key Behavior**
This creates a dynamic system:

Gamepad connected
UI shows:
🎮 Button icons

Keyboard or unknown input
UI shows:
⌨️ Text labels

🧪 **Usage (how to use this in your project)**

Step-by-step:

1. Add component to a GameObject
Attach:
👉 Gamepad Icon Rebind Handler (TMP)

2. Assign rebindControlGroup
Drag the parent object that contains:
- All your rebind UI elements

3. Ensure setup is complete
You must have:
- RebindControlManagerTMP components
- Proper UI references assigned

- InputSystemExtensionData configured

4. Run the game
The script will:
- Automatically hook into all UI elements
- Replace text with icons when possible

⚠️ **Requirements**

1. Icon mappings must exist
If your data doesn't contain a sprite:
👉 It will fallback to text

2. Control paths must match
Example:
- "buttonSouth"
- "dpad/up"

If mismatched:
👉 No icon will be found

3. UI references must be assigned
Each manager must have:
- Text fields
- Image fields

🧭 **How it fits your system**
This is the **runtime visual layer**.

Editor tools
Generate icon mappings

Data asset
Stores mappings

Rebind system
Handles input changes

This script
👉 Converts input data into visuals

🚀 **Final insight**
This script solves a subtle but important UX problem:

Without it:
❌ UI shows "buttonSouth"
❌ Not user-friendly

With it:
✅ Shows actual controller icons
✅ Feels polished and professional
✅ Adapts automatically to device

It's the difference between:
👉 "functional UI"
and
👉 "player-friendly UI"

Reset All Rebind Control

This script is a **simple but very practical UI helper**. Its entire purpose is to give you a **"Reset All" button** that restores every rebinding control in a group back to its original/default input configuration. Instead of resetting one control at a time, this component handles **all of them at once**.

🧠 Core Idea

You attach this component to an object in your UI and assign:

- A **button** (the "Reset All" button).
- A **parent object** that contains multiple rebinding controls.

When the button is pressed → every rebinding control inside that group is reset.

📦 Variables (Fields)

🔷 resetAllButton

- Type: UI Button.
- Purpose: This is the button the player clicks to trigger the reset.

👉 If this is not assigned, the script logs a warning and nothing happens.

🔷 rebindControlGroup

- Type: GameObject (a container).
- Purpose: Acts as a **parent holder** for all rebinding UI elements.

👉 The script will search inside this object and its children to find:

- Standard rebinding components.
- TextMeshPro-based rebinding components.

🔷 rebindControls

- Type: List of standard rebind managers.
- Purpose: Stores all classic (non-TMP) rebinding components found in the group.

👉 Filled automatically at startup.

🔷 rebindControlsTMP

- Type: List of TMP rebind managers.
- Purpose: Stores all TextMeshPro-based rebinding components.

👉 Also filled automatically at startup.

⚙️ Method Behavior

🔷 Start()

This runs when the scene begins.

What it does:

1. Connects the button

- If the button exists:
  - It registers the ResetAll method to be called when the button is clicked.
- If not:
  - Shows a warning in the console.

2. Finds all rebinding components

Inside the rebindControlGroup, it searches for:

- All standard rebind managers → stored in rebindControls.
- All TMP rebind managers → stored in rebindControlsTMP.

👉 This means you don't need to manually assign each control.

Everything is discovered automatically.

🔷 ResetAll()

This is the **main action method**, triggered by the button.

What it does:

1. Reset standard controls

- Loops through every item in rebindControls.
- Calls their reset function.

2. Reset TMP controls

- Loops through every item in rebindControlsTMP.
- Calls their reset function as well.

Important detail:

Each individual rebind manager already knows:

- What binding it controls.
- What the default value is.

So this script simply tells all of them:

👉 "Go back to default."

🔄 **Flow of Execution**

Here's how everything connects:

1. Scene starts.
2. Script:
    - Hooks the button.
    - Scans the UI group.
3. Player clicks **Reset All**.
4. Script:
    - Iterates over all rebinding managers.
    - Calls reset on each one.
5. Each manager:
    - Restores its default binding.
    - Updates its UI automatically.

🎮 **Usage Example**

Typical setup in a settings menu:

```
Settings Menu
├── Controls Panel (rebindControlGroup)
│   ├── Jump Rebind
│   ├── Shoot Rebind
│   ├── Move Rebind
│   └── Pause Rebind
│
└── Reset All Button
```

Setup steps:

1. Add this component to any GameObject.
2. Assign:
    - The **Reset All Button**.
    - The **Controls Panel (group)**.
3. Done.

Now when the player presses the button:

👉 All bindings reset instantly.

💡 **Design Notes**

✓ **Automatic discovery**

You don't need to manually list controls.
This makes it scalable and easy to maintain.

✓ **Supports two systems**

It works with:

- Standard rebind managers.
- TMP-based rebind managers.

✓ **Safe execution**

- Ignores null entries.
- Warns if button is missing.

⚠️ **Limitations / Things to Watch**

- The group **must be assigned**, or nothing will be found.
- If rebinding components are added **after Start**, they won't be included automatically.
- It assumes each manager correctly implements its own reset logic.

🧩 **Summary**

This script acts like a **central reset switch**:

- Collects all rebinding controls inside a group.
- Listens for a button click.
- Resets everything in one action.

👉 It doesn't handle rebinding itself —
it simply coordinates multiple rebinding systems efficiently.

Input Display Manager

This script is a **UI controller for input visualization**.
Its job is to **show the correct icons for player inputs** (keyboard or gamepad) and **animate them when inputs are pressed or released**.
Think of it as the system behind things like:

👉 "Press Ⓐ to jump"
👉 Highlighting WASD arrows when you move
👉 Automatically switching icons when a controller is plugged in

🧠 **Core Idea**

The script manages a set of UI elements (icons) and:

1. Detects **which device is being used** (keyboard or gamepad)
2. Chooses the **correct icon for each input**
3. Listens to input events
4. Animates icons (color changes, fade in/out)

📦 **Main Components**

🎮 **Enum: ControlType**

Defines which input device is active:

- **Keyboard**
- **Gamepad**

🧩 **Data Structures**

These define what will appear on the UI.

🔷 **InputViewerData (Single Input)**

Represents **one input icon**
Variables:

- **nameTag**
  Identifier used to find and control this viewer
- **enable**
  Whether this viewer is active
- **inputActionReference**
  The input action it listens to
- **keyboardId / gamepadId**
  Which binding to use depending on device
- **inputIcon**
  The UI image that displays the icon
- **inputEvent**
  Internal listener that reacts to input

👉 Example:
A “Jump” icon

🔷 **InputMultipleViewsData (Directional Input)**
Represents **multiple icons for directions**
Variables:

- Same base structure as above, plus:
- **inputIconUp / Down / Left / Right**
  Separate icons for each direction
- **inputEvent**
  Handles directional input (like movement stick)

👉 Example:
WASD arrows or D-pad

🎛️ **Inspector Settings**

🔄 **Auto Detection**
automatic

- If true → detects device automatically

controlType

- Default device when auto is off

🎨 **Visual Settings**
activatedColor

- Color when input is pressed

disabledColor

- Color when not pressed

transitionTime

- How fast color changes

hideTransitionTime

- How fast icons fade out

📋 **Data Lists**
inputViewerData

- List of single input icons

inputMultipleViewsData

- List of directional input groups

🔒 **Private Fields**

extensionData

- Stores all input-to-sprite mappings
- Used to get correct icons

colorTransitions

- Tracks active animations per icon
- Prevents overlapping animations

⚙️ **Unity Lifecycle**

Awake
Purpose:
Validate all data

What it checks:

- Missing names
- Missing input actions

- Missing UI images

👉 Prevents broken UI setups

OnEnable
What happens:

1. Detect control type (if automatic)
2. Start listening for device changes
3. Initialize input events (single inputs)
4. Initialize directional inputs
5. Update all icons

👉 This is where everything becomes active

OnDisable / OnDestroy
Calls cleanup:

- Stops listening to devices
- Removes input events
- Stops animations

🔍 **Input Detection**

DetectControlType
Logic:

- If any gamepad is connected → Gamepad
- Otherwise → Keyboard

OnDeviceChange
Triggered when:

- Controller is plugged/unplugged

Behavior:

- Updates control type
- Refreshes icons

👉 This enables seamless switching between keyboard and controller

🎯 **Initialization**

InitializeInputViewerData
Sets up **single input icons**

For each entry:

1. Remove old event
2. Create new input listener

Input behavior:

- On press → change color to active
- On release → change color to inactive

Initial state:

- If enabled → visible
- If disabled → hidden

InitializeInputMultipleViewsData
Sets up **directional inputs**

Behavior differs by device:

🎮 **Gamepad:**

- Only “Up” icon is used
- Highlights when pressed

⌨️ **Keyboard:**

- Activates directions based on input vector:
  - Up → Y > 0
  - Down → Y < 0
  - Left → X < 0
  - Right → X > 0

On release:

- All icons fade back to inactive

🖼️ **Icon Updating**

UpdateIcons
Main refresh method
Calls:

- Update single icons
- Update directional icons

UpdateInputViewerIcons
For each single input:

1. Choose correct binding (keyboard/gamepad)
2. Get sprite
3. Assign to UI

UpdateInputMultipleViewsIcons

Keyboard:

- Finds all direction bindings (Up/Down/Left/Right)
- Assigns each icon separately
- Shows all icons

Gamepad:

- Uses only one icon (Up)
- Hides others

GetSpriteForBinding
Purpose:
Find the correct icon for an input

Keyboard:

- Matches binding path with stored key names
- Returns corresponding sprite

Gamepad:

1. Detects controller type:
   - PlayStation
   - Xbox
2. Picks correct icon set

Fallback:

If nothing matches → default icon

🎛️ View Control

EnableAndDisableViewer
Purpose:
Enable or disable specific UI elements

How it works:
1. Searches by nameTag
2. Updates enable state
3. Applies visibility and animation

Device differences:
- Gamepad → only main icon shown
- Keyboard → all directions handled

HandleIconEnableDisable
Enable:
- Activates object
- Fades in

Disable:
- Fades out
- Deactivates after animation

FadeOutAndDeactivate
Gradually:
- Reduces opacity to zero
- Then disables the object

🎨 Color & Animation

StartColorTransition
- Starts smooth color change
- Cancels previous animation if needed

ColorTransitionCoroutine
- Interpolates color over time

SetInitialColor
- Sets default inactive color
- Ensures object is visible

SetTransparent
- Makes icon invisible
- Disables object

SetDirectionActive
- Activates/deactivates directional icon with animation

🧹 Cleanup

OnClean
Central cleanup method:
- Stops device listening
- Removes input events

- Stops animations

UnbindAllEvents
- Removes all input listeners

StopAllColorTransitions
- Stops all animations
- Clears tracking

🎯 Usage

How to use

1. Add to a GameObject

2. Configure:
- Add input viewer entries
- Assign:
  - Input actions
  - Binding IDs
  - UI images

3. (Optional) Enable automatic detection

4. Run the game

What happens

🎮 Device detection
- Plug controller → icons switch to gamepad
- Remove controller → back to keyboard

🎨 UI behavior
- Press input → icon lights up
- Release → fades back

🧭 Directional input
- Keyboard → multiple arrows light up
- Gamepad → single icon behavior

✅ What This System Solves

✓ Dynamic UI icons
No need to manually swap sprites

✓ Multi-device support
Keyboard + controller handled automatically

✓ Visual feedback
Smooth animations for input states

✓ Scalable system
Supports many inputs and viewers

⚠️ Important Notes

- Requires properly configured input mappings
- Needs sprite database (extensionData)

- Uses coroutines → many icons = more processing

🧩 **Big Picture**
This script is the **visual layer of your input system**:
- Input System → detects actions
- This script → converts them into visuals
- UI → shows feedback to the player

Input Display Manager Inspector

This script is an **editor tool that enhances the inspector** of the input display system.
Its purpose is to **automatically create and organize UI images** based on your input setup, so you don't have to do it manually.

🧠 **Core Idea**
Normally, setting up input UI requires you to:
- Create UI images manually
- Name them correctly
- Assign them to the right fields

This script adds a button that does all of that for you:
👉 It reads your input configuration
👉 It creates (or reuses) UI images
👉 It assigns them automatically

📦 **Main Component**
InputDisplayManagerInspector
This is a **custom inspector**, meaning:
👉 It replaces or extends the default inspector view of another component
In this case:
- It modifies how the **Input Display Manager** appears in the editor

🎛️ **Inspector Interface**

OnInspectorGUI
This is the method that **draws the inspector UI**.

What it does step-by-step:

1. Sync data
- Updates serialized values before drawing

2. Draw button
Creates a button labeled:
👉 **"Create Automatic Presets"**

When clicked:
For each selected object:
1. Record undo state
2. Run automatic preset creation
3. Mark object as modified (so changes are saved)

👉 This supports multi-selection (editing multiple objects at once)

3. Add spacing
Just for visual separation in the inspector

4. Draw default inspector
- Shows all normal fields

- Keeps standard behavior intact

5. Apply changes
- Saves any modifications made in the inspector

⚙ **Preset Creation Logic**

CreateAutomaticPresets
This is the **core function** that generates UI elements.

General idea:
For each input defined in your system:
👉 Ensure there is a corresponding UI image
👉 Ensure it is properly named
👉 Ensure it is assigned

🧍 **Single Input Processing**
Handles simple inputs like:
- Jump
- Interact

Steps:

1. Loop through all entries
Each entry represents one input

2. Skip invalid entries
If no action exists → ignore

3. Set nameTag
- Uses the input action name
- Ensures consistency

4. Define expected name
Example:
Image (Jump)

5. Handle existing image
If already assigned:
- Rename it
- Keep using it

6. If no image assigned
Try to find one in the hierarchy:

Case A: Found existing object
- Reuse it
- Ensure it has an image component
- Rename it

Case B: Not found
- Create a new UI object
- Add required components
- Set default size
- Parent it correctly

7. Assign image back

- Store reference in the data

Directional Input Processing
Handles inputs like:
- Movement (Up, Down, Left, Right)

Steps:

1. Loop through directional entries

2. Skip invalid ones

3. Set nameTag
Same logic as single inputs

4. Create or update 4 images:
- Up
- Down
- Left
- Right

Each uses a helper method:
CreateOrRenameDirectionalImage

Helper Method

CreateOrRenameDirectionalImage
Handles one directional image at a time.

Behavior:

If image already exists:
- Rename it
- Keep using it

If not:

Try to find existing object
If found:
- Reuse it
- Add image component if missing

If not found:
- Create new UI object
- Add required components
- Set default size
- Parent it

Naming format:
Examples:
Image (Move_Up)
Image (Move_Left)

Undo System
Throughout the script, undo is registered for:

- Object creation
- Renaming
- Component changes

👉 This allows you to press **Ctrl+Z** and revert everything safely

🎯 **Usage**

How to use

1. Add Input Display Manager to a GameObject

2. Configure input actions
- Assign input actions in the inspector

3. Click button:
👉 **"Create Automatic Presets"**

What happens

For each input:
- A UI image is created (if missing)
- Existing ones are reused
- Everything is renamed consistently

For directional inputs:
- 4 images are created:
  - Up
  - Down
  - Left
  - Right

All images:
- Are placed as children of the manager object
- Have a default size
- Are ready to use

✅ **Benefits**

✓ **Saves time**
No manual UI setup

✓ **Prevents errors**
No missing references or wrong names

✓ **Consistent structure**
All elements follow the same naming pattern

✓ **Reusable**
Works even if some UI already exists

✓ **Safe**
Undo system protects changes

⚠️ **Limitations**

- Creates basic layout only (no positioning logic)
- Default size may need adjustment

- Assumes a simple hierarchy

🧩 **Big Picture**
This script is a **tooling layer** that sits on top of your UI system:
- Your input system defines actions
- Your display manager shows them
- This script **automates the setup process**

Input Display Controller

This script acts as a **state controller for input display UI elements**.
It doesn't render anything by itself—instead, it tells another system **which input visuals should be ON or OFF** based on a chosen state.

🧠 **Core Idea**
You define **groups of UI elements (by name)** and decide:
- Which ones should be **enabled**
- Which ones should be **disabled**

Then, with a single call, the script applies that configuration.
👉 Think of it like a **preset switch** for input display layouts.

🟫 **Main Concept**
There are two predefined configurations:
- **EntriesToEnable** → what happens when the state is ON
- **EntriesToDisable** → what happens when the state is OFF

And a method that applies one of those sets.

📦 **Data Structure**
🔷 **InputDisplayEntry**
This is a small data container used to describe a single UI element.
Contains:
▪ **nameTag**
- A string identifier.
- Used to find a specific display element inside the system.

👉 Example:
- "Jump"
- "Shoot"
- "Interact"

▪ **enable**
- A boolean value.
- Determines whether that element should be:
  - **true** → enabled (visible/active)
  - **false** → disabled (hidden/inactive)

What this represents:
👉 "For this element (by name), should it be ON or OFF?"

📦 **Variables Explained**

🔷 **entriesToEnable**
- A list of InputDisplayEntry.
- Used when the system is **activated (true state)**.

👉 Defines:
- Which elements should be enabled or disabled when active.

🔷 **entriesToDisable**

- Another list of InputDisplayEntry.
- Used when the system is **deactivated (false state)**.

👉 Defines:

- The opposite configuration.

🔷 **displayManager**

- Reference to another component responsible for actually showing/hiding UI elements.

👉 This script does not manipulate UI directly.
It delegates that responsibility.

⚙️ **Method Behavior**

🔷 **Awake()**
Purpose:
Initialize and validate required dependencies.

What it does:

1. Tries to get the display manager from the same object.
2. If it fails:
    - Logs an error.
    - Disables itself to prevent further issues.

Why this matters:
This script **depends entirely** on the display manager.
Without it, it cannot function.

🔷 **ApplyDisplayState(enable)**
Purpose:
Applies one of the predefined configurations.

🔄 **Step-by-step:**
1. Choose which configuration to use

- If enable is true → use entriesToEnable
- If enable is false → use entriesToDisable

2. Validate the list

- If the list is empty or missing → do nothing

3. Apply each entry
For every entry in the selected list:

- It sends:
    - The element name (nameTag)
    - Whether it should be enabled (enable)

To the display manager.

Important detail:
👉 This script does not know *how* elements are enabled/disabled
👉 It only tells **what should happen**

🔄 **Flow of Execution**

1. Script initializes.
2. Finds the display manager.
3. Another system calls ApplyDisplayState(true or false).
4. Script:
    - Selects the correct list.
    - Iterates through entries.
    - Sends instructions to the manager.
5. Manager updates UI accordingly.

🎮 Usage Scenarios

🟢 1. Switching Input Modes

Example:

- Keyboard mode → show keyboard icons
- Gamepad mode → show controller icons

🟢 2. Context-Sensitive UI

Example:

- In combat → show combat actions
- In menu → hide gameplay controls

🟢 3. Tutorial System

Example:

- Step 1 → show only “Move”
- Step 2 → show “Jump” + “Move”

🟢 4. Accessibility Toggles

Example:

- Enable simplified UI
- Disable advanced controls

🧩 Example Setup

Imagine this configuration:

entriesToEnable

- Jump → enabled
- Shoot → enabled
- Pause → disabled

entriesToDisable

- Jump → disabled
- Shoot → disabled
- Pause → enabled

Result:

- Calling with true → gameplay UI active
- Calling with false → menu UI active

💡 Design Strengths

✓ Data-driven

- No hardcoding.
- Fully configurable in the editor.

✓ Decoupled system

- This script decides **what**
- Manager decides **how**

✓ Flexible

- Can represent any UI state.
- Works with any number of elements.

✓ Reusable

- Same component can be reused across multiple UI contexts.

⚠️ Limitations

❗ **Depends on naming**
- If nameTag doesn't match an existing element → nothing happens.

❗ **No validation per entry**
- It assumes all entries are valid.

❗ **Static configuration**
- Lists are defined beforehand (not dynamic unless modified at runtime).

🧠 **Mental Model**
Think of this script like a **remote control with presets**:
- Button A → apply configuration A
- Button B → apply configuration B

And the display manager is the **TV** that actually changes.

🧩 **Summary**
This script:
- Defines UI states using named entries
- Switches between two configurations
- Delegates execution to a display manager
- Enables/disables UI elements in bulk

Input Debug Logger

This script is a **debugging tool** designed to compare and understand how input behaves across two systems:
- The **old input system** (based on key codes)
- The **new input system** (based on device paths like <Keyboard>/a)

It logs everything in detail so you can **see exactly what happens when you press a key**.

🧠 **Core Idea**
Whenever you press a key, the script:
1. Detects it using the **new input system**
2. Detects it using the **old system**
3. Tries to match both representations
4. Checks if that key exists in your custom database
5. Logs all information (including icons/sprites if available)

📦 **Main Components**

🗺️ **Manual Mapping**
ManualMap
What it is:
A **dictionary (lookup table)**.
Purpose:
Some keys don't match cleanly between the two systems.
This map manually defines those relationships.

Example concept:
- Old system: LeftControl
- New system: <Keyboard>/leftCtrl

Why it's needed:
Not all keys follow predictable naming rules.

Includes:
- Modifier keys (Ctrl, Shift, Alt)
- Special keys (Enter, Escape, etc.)

- Symbols (comma, slash, etc.)
- Mouse buttons

📊 **Field**
data
What it is:
A reference to a **global configuration object**.
Contains:
A list of key definitions:
- KeyCode (old system)
- Path name (new system)
- Sprite (icon)

Purpose:
Allows the script to:
- Check if a key exists in your system
- Find its index
- Check if it has a visual icon

⚙ **Unity Methods**

Start
What it does:
- Loads the global configuration (data)
- If missing → shows warning

Important behavior:
The script **still runs even if data is missing**, but skips validation.

Update
Runs every frame.

It performs 3 steps:

1. Collect new input system presses
Calls:
CollectPressedNewInputPaths

2. Process old system inputs
Calls:
ProcessLegacyInputKeys

3. Log new-only inputs
Calls:
LogNewOnlyInputs

🎮 **Input Collection**

CollectPressedNewInputPaths
Purpose:
Detects all inputs pressed **this frame** using the new system.

It checks:
Keyboard
- Iterates over all keys
- Stores paths like:
- <Keyboard>/a

Mouse

- Left, right, middle, forward, back buttons

Gamepad
- Face buttons (A, B, X, Y)
- D-pad directions

Output:
A list of strings representing pressed inputs.

📋 Input Logging

ProcessLegacyInputKeys
Purpose:
Handles input from the **old system** and compares it with the new one.

How it works:

1. Loop through all possible keys
It checks every key code.

2. Detect key press
If a key was pressed this frame:
- Continue processing

3. Resolve matching new system path
Calls:
ResolveNewPathForKey

4. Lookup in data
If the configuration exists:
It tries two searches:
- **By key code**
- **By path name**

5. Extract info
- Index (by key code)
- Index (by path)
- Stored name
- Whether it has a sprite

6. Build log message
Example structure:
[Pressed]
KeyCode: X
New InputSystem: <Keyboard>/x
Index by KeyCode: 5
Index by Name: 6
Sprite exists: yes/no

7. Print log

8. Exception safety
If something breaks:
- Logs warning instead of crashing

LogNewOnlyInputs
Purpose:
Logs inputs detected **only by the new system**.

Why this matters:
Some inputs:

- Are not detected by the old system
- Or don't map cleanly

Behavior:
For each new input:

1. Check if it exists in data
2. If yes:
    - Log index + sprite info
3. If no:
    - Log as missing

🔍 **Path Resolution**

ResolveNewPathForKey
Purpose:
Convert a key from the old system into a new system path.

It uses multiple strategies:

1. Manual mapping
Checks predefined mappings.

2. Letter keys
A → <Keyboard>/a

3. Number keys (top row)
0–9 → <Keyboard>/0 to <Keyboard>/9

4. Numpad keys
→ <Keyboard>/numpad0 etc.

5. Function keys
F1 → <Keyboard>/f1

6. Heuristic matching
If no direct mapping:

- Compares normalized strings
- Tries to match pressed inputs

7. Fallback
If nothing works:
Unknown mapping

🔧 **Helper Method**

NormalizeForMatch
Purpose:
Prepare strings for comparison.

What it does:

- Converts to lowercase
- Removes non-alphanumeric characters

Example:
"Left Ctrl" → "leftctrl"
"<Keyboard>/leftCtrl" → "leftctrl"

Why:
Makes matching more flexible and forgiving.

🎯 **Usage**

How to use it

1. Add this script to a GameObject
2. Enter Play Mode
3. Press keys/buttons

What you will see
Console logs like:

When pressing a key:
[Pressed] KeyCode: Space | New InputSystem: <Keyboard>/space | ...

When only new system detects:
[New Only] Path pressed: <Gamepad>/buttonSouth | ...

✅ **What This Tool Is Good For**

✓ **Debugging input systems**
Compare old vs new behavior

✓ **Building input databases**
Check if keys exist in your system

✓ **UI systems**
Verify if sprites/icons are assigned

✓ **Detect missing mappings**
Find keys that are not properly configured

✓ **Cross-device testing**
Keyboard, mouse, gamepad

⚠️ **Limitations**

- Runs every frame → can spam logs
- Iterates all key codes → not performance-friendly for production
- Depends on your configuration data for full functionality

🧩 **Big Picture**
This script is like a **translator + inspector** between:

- Old input system
- New input system
- Your custom key database

Test Script

This script is a **simple gameplay test component** that connects the **new input system** to a **physics object**, while also interacting with a UI system that displays inputs.
Think of it as a small sandbox that answers:
👉 "What happens when I press jump or move?"
👉 "Can I see those inputs visually?"

🧠 **Core Idea**
The script does three main things:

1. Listens to **input actions** (jump and movement)
2. Applies those inputs to a **physics object**
3. Optionally controls a **UI display** that shows inputs

📦 **Fields (Variables)**

🎮 **Input Settings**

actionJump

- Represents the **jump input action**
- Expected value type: a number (pressed/not pressed or pressure)

👉 Example:

- Space bar
- Gamepad button

actionMove

- Represents the **movement input action**
- Expected value type: a 2D direction (horizontal + vertical)

👉 Example:

- WASD
- Analog stick

🧱 **Target Settings**

targetRigidbody

- The physics object that will be moved
- Receives velocity changes and forces

👉 This is what actually moves in the scene

speed

- Multiplier for movement speed

👉 Higher value = faster movement

🖥️ **Display Manager**

displayManager

- Reference to a system that shows input UI

👉 Used to show things like:

- “Press E”
- “Jump button”

nameTag

- Identifier used to select which UI element to control

👉 Example:

- "Interact"
- "Jump"

enable

- Determines whether the UI should be turned on or off

⚙️ **Internal Event Variables**

jumpInputEvent

- Holds the configuration for the jump input behavior

moveInputEvent

- Holds the configuration for movement behavior

👉 These act like **listeners** that react to input events.

🔑 **Properties**

These are just **controlled access points** to the fields.
They allow:

- Reading values
- Changing values safely

👉 They don't add new behavior, just structure.

⚙️ **Unity Methods**

Start
This is where everything is configured.

🎯 **Jump Setup**
Creates a listener for the jump action.

When jump is pressed:

1. Reset vertical velocity
   👉 Ensures consistent jump height (prevents stacking upward speed)
2. Apply upward force
   👉 Simulates a jump

Result:
Every time you press jump → the object jumps cleanly.

🎯 **Movement Setup**
Creates a listener for movement input.

While input is held:

1. Read direction (X and Y)
2. Multiply by speed
3. Apply it to horizontal movement
4. Keep vertical velocity unchanged

👉 This means:

- You can move while jumping/falling
- Gravity is preserved

When input is released:

- Horizontal movement is stopped
- Vertical movement continues

👉 So:

- You stop moving sideways
- But still fall naturally

🧹 **OnDestroy**
Purpose:
Clean up event listeners

What it does:

- Removes subscriptions
- Frees memory

👉 Important because:
Event systems can keep references alive if not cleared

🔧 **Public Method**

EnableAndDisableViewer
Purpose:
Control the visibility of input UI

What it does:

- Uses the display manager
- Targets a specific UI element using nameTag
- Enables or disables it based on enable

Extra detail:
It has a context menu option, meaning:
👉 You can trigger it directly from the inspector

🎯 **Usage**

How to use this script

1. Add it to a GameObject

2. Assign:

- Jump action
- Movement action
- Rigidbody (object to move)
- Display manager (optional)

3. Press Play

What happens in-game

🎮 **Movement**

- Move input → object moves horizontally
- Release → stops moving

🦘 **Jump**

- Press jump → object jumps upward

🖥️ **UI**

- You can toggle input display visibility via:
  - Inspector button
  - Or code

✅ **What This Script Demonstrates**

✓ **Basic input handling**
Using action-based input instead of raw key checks

✓ **Event-driven design**
Instead of checking input every frame, it reacts to events

✓ **Physics interaction**
Applies velocity and force properly

✓ **Separation of concerns**

- Input logic
- Movement logic
- UI control

⚠ **Important Notes**

Movement replaces velocity directly

- No acceleration
- No smoothing

👉 Good for testing, not final gameplay

Jump uses fixed force

- No scaling based on input

No ground check

- You can jump infinitely

👉 This is intentional — it's a **test script**, not a full controller

🧩 **Big Picture**

This script is a **bridge between input and gameplay**:

- Input System → triggers events
- Events → control physics
- Physics → moves object
- UI → optionally reflects input